

A Certified Admission Gate for Online Reconfiguration of Time-Sensitive Networks

Huyen-Trang Le^{id}

IBISC Laboratory

Université d'Évry Paris-Saclay

Évry-Courcouronnes, France

huyen.le@etud.univ-evry.fr

Samia Bouzefrane^{id}

CEDRIC Laboratory

Conservatoire National des Arts et Métiers (Cnam)

Paris, France

samia.bouzefrane@lecnam.net

Abstract—Time-Sensitive Networking (TSN) guarantees worst-case latency only for a configuration that has been fixed and verified offline. When flows join a running network, fast optimisers (heuristics, ILP/SMT solvers, learned policies) can propose new configurations, but none of them certifies that every deadline still holds. We place a *certified admission gate* between any such proposer and the network. The gate accepts a flow only together with a certificate: a per-hop witness of the network-calculus bound of every affected flow, which an independent checker of about fifty lines re-verifies without trusting the analysis code. We model 1 Gbit/s switches with non-preemptive static priority and token-bucket flows, and bound delays with Total Flow Analysis. An incremental variant of the gate recomputes only the ports a request can affect and yields bit-identical bounds. On line and tree topologies with 1,000 seeded requests and 10 seeds, our bounds match the reference tool *panco* to a relative difference of 3.3×10^{-16} per hop and 2.9×10^{-6} end to end, no simulated delay exceeds its bound across 2,118 admitted flows, and the incremental gate is 1.9–2.0× faster than full recomputation. All code, data and figures are reproducible from a public repository.

Index Terms—Time-Sensitive Networking, network calculus, admission control, certification, worst-case delay

I. PROBLEM

Factories, vehicles and aircraft increasingly replace dedicated fieldbuses with a single Ethernet network shared by control loops, sensors, video and best-effort traffic. For control traffic, a message that arrives late is not a slower service but a failure: a robot arm or a braking system acts on stale data. What these systems need from the network is therefore not a good average latency but a *guaranteed worst case*, and that guarantee must survive while the system evolves, as machines, sensors and applications are added during operation.

Time-Sensitive Networking (TSN), a set of IEEE 802.1Q mechanisms [1], provides such bounded latency on switched Ethernet. Its guarantees rest on an analysis of the whole flow set: a worst-case delay bound computed with network calculus [2], [3] or a schedule synthesised offline [4]. Real deployments, however, are not static. Flows are added while the network runs, and every change can increase the delay of flows already admitted, including flows on other paths whose traffic shares a port.

Many tools can *propose* a change quickly: greedy heuristics, ILP or SMT schedulers [4], and, increasingly, deep

reinforcement learning policies. Their outputs are optimised, not proven. A learned policy in particular gives no worst-case guarantee, and even an exact solver is only as trustworthy as its encoding and implementation. For safety-critical traffic, the question at reconfiguration time is binary and must be answered with evidence: *after this change, does every admitted flow still meet its deadline?*

We answer it with an *admission gate*, placed between any proposer and the network, in the spirit of a safety shield for learned controllers [5]. The gate receives one request at a time and returns a verdict together with a *certificate*. A change is committed only if the certificate passes an independent checker. Fig. 1 shows the setting and the pipeline. The guarantee is a safety property relative to the network model: it holds as far as the model of Section II describes the real network.

Contributions. (i) A gate $\text{admit}(S, f) \rightarrow (\text{verdict}, \text{certificate})$ for non-preemptive static-priority TSN, whose certificates are checked by a small, separate checker with an explicit soundness argument (Section III). (ii) An incremental gate that recomputes only the ports a request can affect and provably returns the same bounds as full recomputation. (iii) A validation against the reference tool *panco* [6], a packet-level simulator and unit tests, with all results reproducible (Section IV). (iv) An explicit list of what the guarantee does not cover (Section V).

II. MODEL

Network. The network is a set of switches and end stations joined by full-duplex links of rate $C = 1$ Gbit/s. Each direction of a link is an output *port*, served by the transmitting node. A switch adds a constant processing delay t_{proc} . Routing is fixed: each source–destination pair uses one shortest path, and end stations do not forward. When the network is built, we compute the dependency graph between ports induced by all possible routes and require it to be acyclic (*feed-forward*); its topological order is reused for every flow set.

Flows. A flow f has a source, a destination, a token-bucket arrival curve $\alpha_f(t) = b_f + r_f t$ at its source, a maximum frame size $L_f \leq b_f$, a priority class k_f (0 is the highest) and an end-

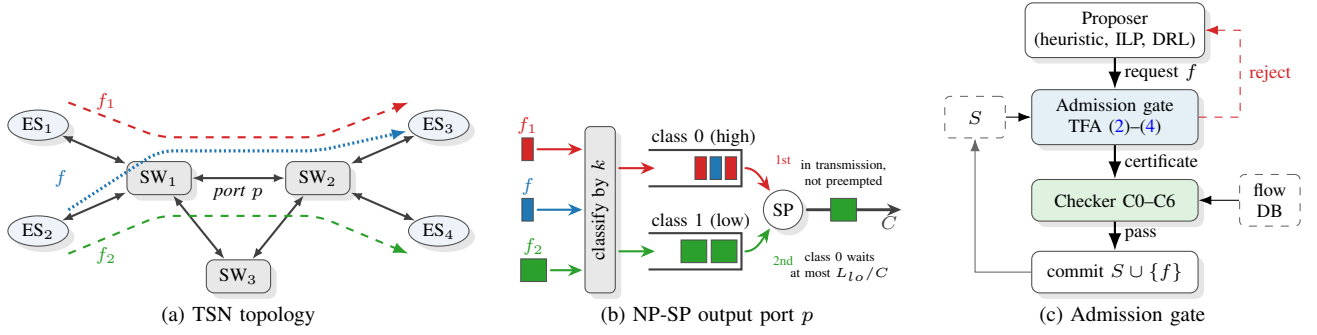


Fig. 1. System architecture. (a) Admitted flows f_1 (class 0) and f_2 (class 1) and a new class-0 request f share output port p . (b) At p , frames are classified by priority into FIFO queues; the strict-priority (SP) selector serves class 0 first but cannot preempt a class-1 frame already in transmission, which is the L_{lo} term of (2). (c) A proposed change is committed only if the gate's certificate passes the independent checker, which also reads the flow database (C0); a rejection returns the violated flows to the proposer.

to-end deadline $D_f^{\max}$. Ports use non-preemptive static priority (NP-SP), FIFO within a class.

Per-hop bound. Consider class k at a port p . Let B_H, R_H be the sums of input bursts and rates of the flows of higher priority at p , B_k, R_k the same sums for class k , and L_{lo} the largest frame of lower priority. Because transmission is not preempted, one lower-priority frame can block class k ; the higher classes take at most $B_H + R_H t$. Class k therefore receives the residual rate-latency service curve

$$\beta_k(t) = (C - R_H) \left[t - \frac{B_H + L_{lo}}{C - R_H} \right]^+, \quad (1)$$

and, as the class aggregate is constrained by $B_k + R_k t$, the delay of any class- k frame at p is bounded by the horizontal deviation [3], [7]

$$d_k = \frac{B_H + L_{lo} + B_k}{C - R_H} + t_{proc}, \quad \text{valid if } R_H + R_k \leq C. \quad (2)$$

If the rate condition fails, the port is unstable and $d_k = \infty$.

Propagation. A flow that leaves a server with delay bound d has output arrival curve $\alpha(t + d)$, so its burst at the next hop is

$$b' = b + r_f d_k. \quad (3)$$

Total Flow Analysis (TFA) visits ports in topological order, applies (2) with the propagated bursts (3), and bounds the end-to-end delay of f along its path $\mathcal{P}_f$ by

$$D_f = \sum_{h \in \mathcal{P}_f} d_{k_f}^{(h)}. \quad (4)$$

A flow set is *schedulable* if every port is stable and $D_f \leq D_f^{\max}$ for every flow.

III. THE ADMISSION GATE

Interface and invariant. The gate holds the admitted set S . On a request f , it analyses $S \cup \{f\}$ and accepts if and only if that set is schedulable. Invariant: *every flow in S meets its deadline under the model*. A request is rejected not only when f itself would miss its deadline, but also when admitting f would make an already admitted flow miss its own; the rejection lists every violated flow and reason.

Certificate. An acceptance carries, for each flow, a *witness*: its parameters, its path, and for each hop the input burst b_{in} , the aggregates $(B_H, R_H, L_{lo}, B_k, R_k)$ and the delay d , plus the end-to-end bound. It also carries a fingerprint of the admitted set before and after the request: the XOR of a SHA-256 hash of each flow, which is order-independent and updated in $O(1)$.

Checker. The checker receives the topology, the witnesses and the operator's flow database, and re-verifies seven conditions without calling the analysis code:

- C0 there is exactly one witness per admitted flow, with that flow's own parameters;
- C1 each witnessed path is the fixed route of the flow;
- C2 the first input burst is at least b_f ;
- C3 each next input burst is at least $b_{in} + r_f d$ (Eq. (3));
- C4 at each port, the claimed aggregates are at least the sums (or maximum, for L_{lo}) recomputed from all witnesses crossing that port;
- C5 $R_H < C$, $R_H + R_k \leq C$, and d is at least the right-hand side of (2);
- C6 the end-to-end bound is at least the sum of hop delays and at most the deadline.

Proposition 1. *If a certificate passes C0–C6, every admitted flow meets its deadline under the model of Section II.*

Proof sketch. The bound (2) is non-decreasing in B_H, R_H (for $R_H < C$), L_{lo} and B_k . We show by induction over the topological order of ports that every witnessed input burst over-approximates the true one. At the first hop this is C2. Assume it holds for all flows entering port p . By C0 and C1, the witnesses at p are exactly the flows that cross p , so by C4 the claimed aggregates over-approximate the true ones; by monotonicity and C5 the claimed d bounds the true delay at p . The true output burst is then at most $b_{in} + r_f d$, which by C3 is at most the next witnessed input burst. Feed-forwardness makes the induction well founded. Summing the per-hop bounds, C6 gives a true end-to-end delay at most the claimed bound, which is at most the deadline. $\square$

C0 is needed: omitting a flow from the certificate only

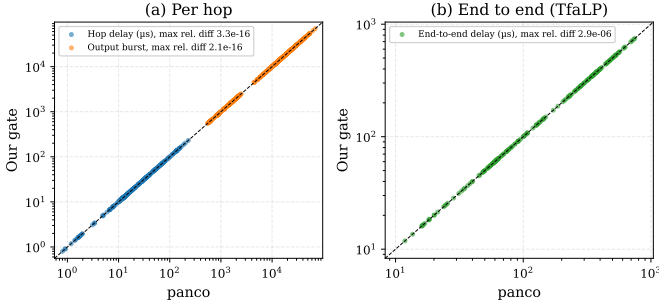


Fig. 2. Bounds of our gate against panco [6], per hop (837 values) and end to end (210 values, TfaLP).

lowers the aggregates of the others, so all remaining conditions would still pass. Comparing with the flow database is the only way to detect the omission, and our tests confirm that the checker rejects it only when the database is supplied. The checker is about fifty lines of code, separate from the analysis, and uses a relative tolerance of 10^{-9} for floating-point comparisons.

Full and incremental gates. The *full gate* reruns TFA over $S \cup \{f\}$ for every request. The *incremental gate* keeps the per-port results, bursts and bounds of S in a cache. For a request, it pushes the ports of the new path into a priority queue keyed by topological index, re-evaluates each popped port, and enqueues the next port of a flow only if that flow’s output burst changed. It writes into overlays, so the cache is untouched until the verdict, and it issues a *delta certificate* that holds witnesses only for flows whose bounds changed. Both gates call the same pure port function, which sums with exactly rounded summation (`math.fsum`) and is therefore independent of the order of flows. By induction over the topological order, both gates feed it identical inputs at every port, so they return the same verdicts and *bit-identical* bounds, not merely close ones.

IV. RESULTS

Setup. We use two feed-forward topologies with 48 ports each: a *line* of 5 switches with 4 end stations per switch, and a *tree* with one core switch and 4 edge switches with 5 end stations each; $C = 1$ Gbit/s and $t_{proc} = 1 \mu s$. Requests are drawn between random end-station pairs from two classes. Class 0 (control, probability 0.3) has 64–256 B frames, periods of 0.25–1 ms and deadlines of 0.1–0.5 ms. Class 1 (stream) has 500–1500 B frames, bursts of one or two frames, periods of 0.125–1 ms and deadlines of 0.5–2 ms. Each run submits 1,000 requests in sequence; we use 10 seeds per topology and run both gates on the same sequence. The code is single-threaded CPython 3.14.

Agreement with panco. We compare our analysis with panco [6] on five scenarios: a hand-computed two-flow example and two seeds of 60 requests on each topology, with $t_{proc} = 0$. Per hop, panco’s static-priority residual server gives the delay and output burst that we compare with (2) and (3). End to end, panco builds one FIFO network per class

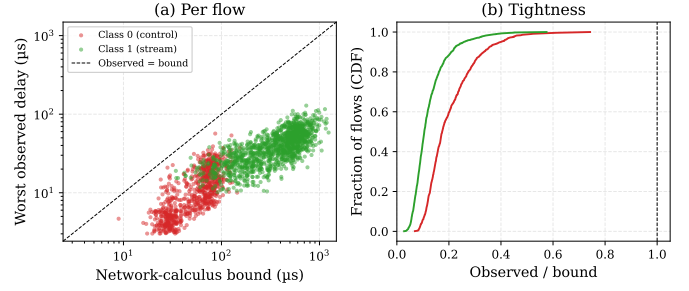


Fig. 3. Worst simulated delay against the bound for 2,118 admitted flows (10ms simulated, line and tree, 10 seeds). (a) Per flow; (b) CDF of observed/bound.

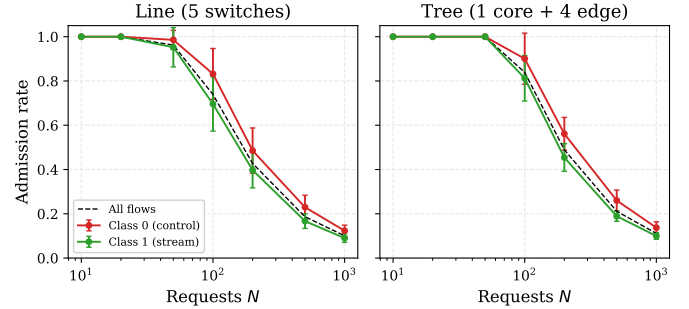


Fig. 4. Fraction of the first N requests accepted, per class (mean $\pm$ std over 10 seeds).

from residual servers and solves TFA as a linear program, independently of our propagation code. The 290 hop delays and 547 output bursts agree to a relative difference of at most 3.3×10^{-16} , that is, to floating-point precision; the 210 end-to-end bounds agree to 2.9×10^{-6} , the precision at which `lp_solve` prints its results (Fig. 2).

Simulation. A discrete-event, packet-level simulator replays every admitted set: each flow emits its burst at a random offset and then frames at rate r_f , and ports serve NP-SP queues frame by frame. Over 2,118 admitted flows, no observed delay exceeds its bound (Fig. 3). The median ratio of observed delay to bound is 0.17 for control traffic and 0.11 for stream traffic, with maxima of 0.74 and 0.58. These ratios are a sanity check, not a measure of tightness: the simulator does not search for the worst-case alignment of bursts.

Admission rate. Both networks accept every request up to about 50, then saturate (Fig. 4). After 1,000 requests, 100 ± 20 flows are admitted on the line and 111 ± 16 on the tree, an overall rate of 10–11 %. Control traffic is accepted slightly more often, as its priority shields it from most interference.

Run time. Over all runs, the full and incremental gates return identical verdicts, and the final bounds of both are equal bit for bit. The incremental gate is $2.02\times$ faster on the line and $1.91\times$ on the tree over 1,000 requests (Table I), and its cost per request grows more slowly with the number of admitted flows (Fig. 5b). The gain is modest because these networks are small: a request still re-evaluates a median of 19 of the

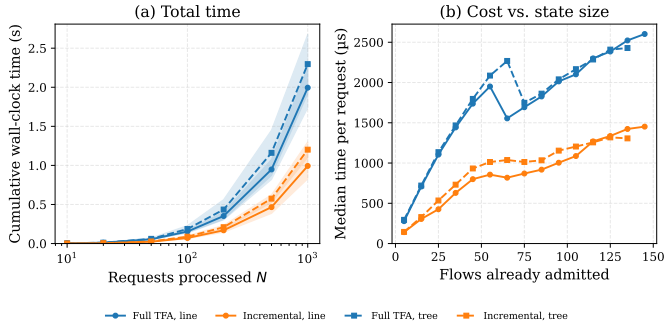


Fig. 5. Run time of the full and incremental gates. (a) Cumulative time over N requests; (b) median time per request against the number of admitted flows.

TABLE I
ADMISSION TIME PER REQUEST (μ s), 10 SEEDS $\times$ 1,000 REQUESTS.

Topology	Gate	Median	p95	Speed-up
Line	Full	1944	2745	
	Incremental	964	1587	2.02 ± 0.13
Tree	Full	2202	3718	
	Incremental	1186	1750	1.91 ± 0.23

48 ports. A median decision takes about 1 ms in Python.

Tests. Eighteen unit tests cover a hand-computed example (to 10^{-12}), gate equality, the simulator and tampered certificates, rejected by C5 (halved delay) and C0 (omitted flow).

V. LIMITATIONS

Model. The guarantee is only as good as the model. We assume a single link rate, fixed shortest-path routing, feed-forward port dependencies and strict NP-SP. Credit-based and time-aware shapers, frame preemption, redundancy and clock effects are not modelled, and Ethernet overheads (preamble, inter-frame gap) must be folded into L_f , b_f and r_f by the operator. The processing delay is a constant upper bound.

Analysis. TFA bounds each port separately and propagates bursts hop by hop. It is sound but pessimistic compared with methods that exploit the pay-burst-only-once effect, such as PMOO or LP-based analyses [7], [8]. Pessimism lowers the admission rate; it never admits an unsafe flow.

Certification. The soundness argument is a proof sketch on paper, not a mechanised proof, and the checker compares floating-point values with a relative tolerance of 10^{-9} , which the argument does not account for. The checker is small but not itself verified; a formally verified checker, or exact rational arithmetic, would close both gaps. The certificate attests to the model, not to the configuration actually deployed on the switches.

Scope of the evaluation. The gate only handles new flows; removing a flow and reconfiguring several flows at once are not implemented. The networks are small, saturate at about 100 flows, and only two classes are used, so the measured speed-up of the incremental gate may not carry over to larger networks. The simulator does not reach the worst case, and the

run times are those of single-threaded Python on one machine. Finally, no proposer is connected yet: the gate has been tested on random requests only, not behind a heuristic or a learned policy.

VI. CONCLUSION

Contribution. This work makes online traffic allocation in TSN safe to automate. Each time a flow asks for a share of the network, the gate decides whether the ports on its path can still absorb its burst and rate without any admitted flow missing its deadline, and allocates the resources only if they can. Every allocation comes with a certificate that a fifty-line checker re-verifies (C0–C6), so a proposer, even a learned one, can never commit an unsafe allocation under the model. The incremental gate makes this decision in about 1 ms with bounds bit-identical to full recomputation, and the bounds match panco to numerical precision and hold for all 2,118 simulated flows.

Future directions. Four directions follow. (i) *Capacity*: TFA is pessimistic and caps admission at about 100 flows on our topologies; tighter analyses that keep checkable certificates would let the same network carry more traffic. (ii) *Coverage*: credit-based and time-aware shapers and frame preemption must be modelled to allocate traffic in real TSN deployments. (iii) *Trust*: a mechanised soundness proof and an exact-arithmetic checker would remove the remaining gaps in the guarantee. (iv) *Allocation policy*: supporting flow removal and batched changes, then placing the gate as a shield behind heuristic, ILP and DRL proposers, will show how much admission capacity certification costs each of them.

REFERENCES

- [1] IEEE, “IEEE standard for local and metropolitan area networks—bridges and bridged networks,” IEEE Std 802.1Q-2022, 2022, doi: [10.1109/IEEESTD.2022.10004498](https://doi.org/10.1109/IEEESTD.2022.10004498).
- [2] R. L. Cruz, “A calculus for network delay, Part I: Network elements in isolation,” *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 114–131, 1991, doi: [10.1109/18.61109](https://doi.org/10.1109/18.61109).
- [3] J.-Y. Le Boudec and P. Thiran, *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*, ser. Lecture Notes in Computer Science. Springer, 2001, vol. 2050, doi: [10.1007/3-540-45318-0](https://doi.org/10.1007/3-540-45318-0).
- [4] C. Xue, T. Zhang, Y. Zhou, M. Nixon, A. Loveless, and S. Han, “Real-time scheduling for 802.1Qbv time-sensitive networking (TSN): A systematic review and experimental study,” in *Proc. IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2024, pp. 108–121, doi: [10.1109/RTAS61025.2024.00017](https://doi.org/10.1109/RTAS61025.2024.00017).
- [5] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, doi: [10.1609/aaai.v32i1.11797](https://doi.org/10.1609/aaai.v32i1.11797).
- [6] A. Bouillard, “Trade-off between accuracy and tractability of network calculus in FIFO networks,” *Performance Evaluation*, vol. 153, p. 102250, 2022, doi: [10.1016/j.peva.2021.102250](https://doi.org/10.1016/j.peva.2021.102250).
- [7] A. Bouillard, M. Boyer, and E. Le Corronc, *Deterministic Network Calculus: From Theory to Practical Implementation*. Wiley-ISTE, 2018, doi: [10.1002/9781119440284](https://doi.org/10.1002/9781119440284).
- [8] S. Bondorf and J. B. Schmitt, “The DiscoDNC v2: A comprehensive tool for deterministic network calculus,” *EAI Endorsed Transactions on Internet of Things*, vol. 1, no. 4, p. e4, 2015, doi: [10.4108/icst.valuetools.2014.258167](https://doi.org/10.4108/icst.valuetools.2014.258167).